

파이썬 워크 >



언제 내려? (지하철 하차 알림 시스템)

> 우희원, 김예은

1. 주제 선정

2. 구성

3. 코드 작성

4. 시뮬레이션

5. 향후 보완 및 응용 방향

6. 느낀점

S 주제 선정



S 주제 선정

1시간 37분

오후 1:34 - 3:12 · 2,150원

8분

1

9분

3분

7

1시간 8분

빠른 도착

1시간 25분

오후 1:30 - 2:55 · 2,150원

8분

1

6분

3분

공

24분

5분

5

27분

3분

7

1분



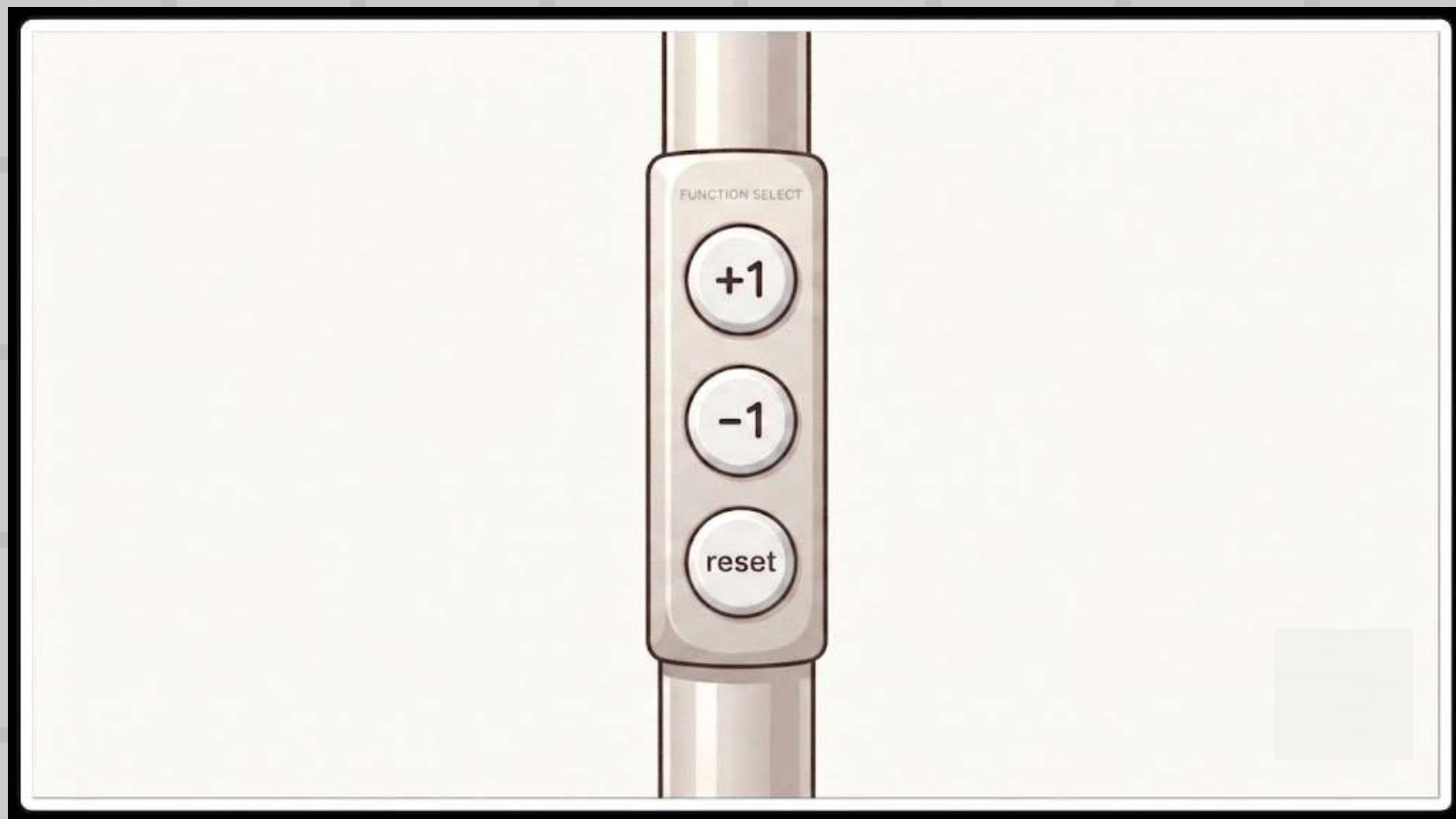
S

구성



S

구성



S 코드 작성

은행 데이터를 연동시켜 구축하는 것은
현실적으로 어렵다 판단

Flask를 사용한 웹 시뮬레이션에 초점을 둠

Flask란?

파이썬 기반의 마이크로 웹 프레임워크
열차 위치 및 승객 상태 등 모든 데이터를 중앙에서 관리하고,
단순히 API 요청을 보내 최신 상태를 받아오기 때문에
이번 프로젝트에 최적화된 도구임

S 코드 작성

```
import os
from threading import Lock
from time import monotonic

from flask import Flask, jsonify, render_template, request

app = Flask(__name__)

STATIONS = [
    "장암", "도봉산", "수락산", "마들", "노원", "중계", "하계", "공릉", "태릉입구",
    "먹골", "중화", "상봉", "면목", "사가정", "용마산", "중곡", "어린이대공원",
]

TRAVEL_SECONDS = 6 # 실제 60초 이동을 10배 빠르게 시연합니다.
state_lock = Lock()
state = {
    "seat": {"id": 1, "occupied": False, "remaining": 0},
    "train": {"station": 0, "direction": 1, "exited": 0, "running": True, "last_tick": monotonic}
}

def status_for(seat):
    """Return the text badge and CSS class from Python-managed seat data."""
    if not seat["occupied"]:
        return {"label": "빈 좌석", "class_name": "normal"}
    if seat["remaining"] == 0:
        return {"label": "하차 알림", "class_name": "exit"}
    if seat["remaining"] <= 2:
        return {"label": "하차 임박", "class_name": "urgent"}
    if seat["remaining"] <= 5:
        return {"label": "준비 상태", "class_name": "ready"}
    return {"label": "일반 상태", "class_name": "normal"}
```

Python 웹 프레임 워크
Flask 사용

시뮬레이션 주기를 10배속 하여
6초마다 한 역씩 이동하도록 구축

S 코드 작성

```
def move_one_station():
    """Move the train once, decrementing the configured passenger count."""
    seat, train = state["seat"], state["train"]
    if seat["occupied"] and seat["remaining"] > 0:
        seat["remaining"] -= 1
    if train["station"] == len(STATIONS) - 1:
        train["direction"] = -1
    elif train["station"] == 0:
        train["direction"] = 1
    train["station"] += train["direction"]

def advance_elapsed_time():
    """Advance all simulation ticks that have elapsed while the train runs."""
    train = state["train"]
    if not train["running"]:
        return
    now = monotonic()
    while now - train["last_tick"] >= TRAVEL_SECONDS:
        move_one_station()
        train["last_tick"] += TRAVEL_SECONDS
```

네트워크가 동기화 되어도 영향이 없는 **monotonic 함수**를
사용해 설정한 6초가 지나면 자동으로 역 이동함

S 코드 작성

```
def display_message():
    """Build the personal seat notification message on the server."""
    seat, train = state["seat"], state["train"]
    if not seat["occupied"]:
        return "하차 예정 정보를 설정해주세요."
    if seat["remaining"] == 0:
        return "이번 역 하차 알림"
    if seat["remaining"] == 1:
        next_index = train["station"] + train["direction"]
        next_station = STATIONS[next_index] if 0 <= next_index < len(STATIONS) else STATIONS[train["station"] - 1]
        return f"다음 역 하차 예정 : 다음 역 {next_station}"
    return f"하차까지 {seat['remaining']}개 역 남음 · 현재 {STATIONS[train['station']]}"
```

남은 역 수를 저장하는 변수로
전광판 문구 지정

S 코드 작성

```
@app.post("/api/action")
def update_state():
    data = request.get_json(silent=True) or {}
    action = data.get("action")
    with state_lock:
        advance_elapsed_time()
        seat, train = state["seat"], state["train"]
        if action == "seat":
            command = data.get("command")
            if command == "toggle":
                if not seat["occupied"]:
                    seat["occupied"] = True
                elif seat["remaining"] == 0:
                    seat.update(occupied=False, remaining=0)
                    train["exited"] += 1
            else:
                seat.update(occupied=False, remaining=0)
            elif command == "plus" and seat["occupied"]:
                seat["remaining"] = min(30, seat["remaining"] + 1)
            elif command == "minus" and seat["occupied"]:
                seat["remaining"] = max(0, seat["remaining"] - 1)
            elif command == "reset" and seat["occupied"]:
                seat["remaining"] = 0
        else:
            return jsonify(error="Unknown seat command"), 400
        elif action == "set-seat":
            try:
                seat["id"] = max(1, min(8, int(data.get("seat_id"))))
            except (TypeError, ValueError):
                return jsonify(error="Seat number must be between 1 and 8"), 400
        elif action == "toggle-running":
            train["running"] = not train["running"]
            train["last_tick"] = monotonic()
        elif action == "restart":
            state["seat"] = {"id": 1, "occupied": False, "remaining": 0}
            state["train"] = {"station": 0, "direction": 1, "exited": 0, "running": True, "las
```

+버튼과 -버튼을 이용해
변수값을 |만큼 증가시키거나 감소시켜 저장함
reset 버튼을 누를 시
최초의 기본값으로 초기화 함

S 시뮬레이션

1

현재 용마산역

용마산

종곡

어린이대공원

3

하차 안내

하차까지 2개 역 남음 · 현재 용마산역

SEAT 01 · 동의 승객

승객 탑승

하차 임박

2개 역

+ 1

- 1

reset

실시간 지하철 위치 및 도착 공공 API를
파이썬 서버에 연동하여 실제 지하철이
역을 통과할 때마다 자동으로 갱신되도록 고도화 가능



느낀점

자유 주제로 해보는 만큼 현실 구현은 어렵더라도
시뮬레이션으로 돌려볼만한 재밌는 주제로 하고 싶었습니다.

평소 실제로 겪는 고충이기도 하였고,
누구나 한 번쯤 상상으로도 해봤을 법한 주제였기에
구상하는 내내 재미있었습니다.